

Failure (a real rollback trigger) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper Python), **Advanced** (how a real rollback trigger avoids false alarms and repeat-firing). Read Basic first even if you already know Python — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

A rollback trigger watches recent test scores as they come in, and decides: "has quality dropped enough, for long enough, that we should go back to the last version we know was good?" A single bad score isn't enough on its own — you're watching the *trend* of the last several scores, not any one of them.

Things to use:

- A list that keeps growing as new scores come in.
- List slicing, `scores[-5:]`, to grab just the most recent few.
- A simple average: `sum(...) / len(...)`.
- An `if` comparing that average to a starting ("baseline") score.

Intermediate Version

The shape of this exercise is: keep a running list of scores, and after every new one arrives, look at just the last `window_size` of them (a "sliding window") and compare their average against a baseline — the score you'd expect a healthy version to keep hitting. If the recent average has fallen far enough below the baseline — say, dropped by 20% or more — that's the trigger firing.

Percent drop is `(baseline - avg_recent) / baseline`. A baseline of `0.95` and a recent average of `0.60` gives `(0.95 - 0.60) / 0.95 ≈ 0.37`, a 37% drop — well past a `0.2` (20%) threshold.

The exact pieces:

- `scores_so_far[-window_size:]` — Python's negative-index slicing grabs the last `window_size` items of a list, however many there are (it won't error even if the list is shorter than `window_size`, which matters for the next point).
- A guard for not enough data yet: `if len(scores_so_far) < window_size: return False` — you can't trust a 2-score average to represent "the last 5 requests" if only 2 have happened.
- `percent_drop = (baseline - avg_recent) / baseline`, then `percent_drop >= drop_threshold`.
- A `rollback_to_previous()` function — for this exercise, a stub that just prints what it would do and returns the version name it reverted to; in the Build Task, this is the real function that rewrites the live pointer.

Advanced Version

Two real problems show up the moment you actually simulate a bad release, instead of just writing the `if` and trusting it:

Flapping — once the trigger has fired and you've called `rollback_to_previous()`, the *next* score you check is still going to be part of a low-scoring recent window (the bad scores don't vanish from the list), so a trigger with no memory of "I already acted on this" will fire again on every subsequent check, calling `rollback_to_previous()` repeatedly for the same decline. A real rollback should fire once per decline, not once per request after the decline starts.

Not enough data yet — this is the same principle as `canary_release`'s minimum sample size: a "recent average" built from only 1 or 2 scores isn't trustworthy evidence of anything, good or bad. The window guard from Intermediate already handles this, but it's worth naming explicitly as the same idea showing up again.

The extra piece needed for the flapping problem: **track whether a rollback has already been triggered for the current decline**, and only reset that once a *new* version is deployed (a real "we're trying again" moment). The cleanest way to hold that piece of state alongside the score history is a small class instead of a bare function — a `RollbackMonitor` that remembers its own scores and whether it's already fired.

- A class with `__init__(self, baseline, window_size, drop_threshold)` that also sets `self.already_rolled_back = False`.
- A method, `record_score(self, score)`, that appends the score, checks the window (only if `not self.already_rolled_back`), and — if it should trigger — sets `self.already_rolled_back = True` before returning `True`, so the very next call sees the flag and does nothing further.

Sketch `RollbackMonitor` yourself before checking Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic names the sliding-window average idea and the comparison. Intermediate shows the real percent-drop math and the not-enough-data guard, plus a stub `rollback_to_previous()` to call. Advanced adds what only shows up once you actually run a full simulated sequence of scores instead of testing one call in isolation — a trigger with no memory fires over and over for the same decline, so it needs to remember it already acted, and stop firing again until a fresh deploy gives it a reason to reset.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
function should_rollback(scores_so_far, baseline, window_size, drop_threshold) ->
  if fewer than window_size scores so far: return False
  recent = the last window_size scores
  avg_recent = average of recent
  percent_drop = (baseline - avg_recent) / baseline
  return percent_drop >= drop_threshold
```

feed a list of simulated scores one at a time into `should_rollback`
the moment it returns `True`, call `rollback_to_previous()` and print when it happens

Here's almost the whole thing — just try running it and reading it line by line:

```
def should_rollback(scores_so_far, baseline, window_size=5, drop_threshold=0.2):
    if len(scores_so_far) < window_size:
        return False
    recent = scores_so_far[-window_size:]
    avg_recent = sum(recent) / len(recent)
    percent_drop = (baseline - avg_recent) / baseline
    return percent_drop >= drop_threshold

def rollback_to_previous():
    print("ROLLBACK: reverting to last known-good version")
    return "v1"

scores = [0.95, 0.94, 0.96, 0.93, 0.95, 0.60, 0.55, 0.58, 0.52, 0.50]
seen = []
for i, s in enumerate(scores):
    seen.append(s)
    if should_rollback(seen, baseline=0.95):
        print(f"request {i}: score {s} -- triggering rollback")
        rollback_to_previous()
```

Expected output: rollback fires (and keeps firing) once the recent-window average drops far enough — run it and find exactly which request number it first triggers on.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
function should_rollback(scores_so_far: list[float], baseline: float, window_size: int) -> bool:
    if not enough scores yet: return False
    recent = last window_size scores
    avg_recent = sum(recent) / len(recent)
    percent_drop = (baseline - avg_recent) / baseline
    return percent_drop >= drop_threshold

function rollback_to_previous() -> str:
    print what happened
    return the version name rolled back to
```

```
def should_rollback(
    scores_so_far: list[float],
    baseline: float,
    window_size: int = 5,
    drop_threshold: float = 0.2,
) -> bool:
    if len(scores_so_far) < window_size:
        return False
    recent = scores_so_far[-window_size:]
    avg_recent = sum(recent) / len(recent)
    percent_drop = (baseline - avg_recent) / baseline
    return percent_drop >= drop_threshold

def rollback_to_previous() -> str:
    print("ROLLBACK: reverting to last known-good version")
    return "v1"
```

Now write the loop that feeds a simulated sequence of scores through `should_rollback`, and notice it keeps calling `rollback_to_previous()` on every request after the first trigger — that's the flapping problem Hint 1's Advanced section named. Compare against the [Solution](#) once you've seen it happen yourself.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

```
class RollbackMonitor:
    __init__(self, baseline, window_size, drop_threshold):
        store all three, plus self.scores = [], self.already_rolled_back = False

    record_score(self, score) -> bool:
        append score to self.scores
        if self.already_rolled_back: return False    # already acted, stay quiet
        if not enough scores yet: return False
        recent = last window_size scores
        avg_recent = average of recent
        percent_drop = (baseline - avg_recent) / baseline
        if percent_drop >= drop_threshold:
            self.already_rolled_back = True
            return True
        return False
```

Here's almost the whole thing — fill in `record_score` 's body yourself:

```
class RollbackMonitor:
    def __init__(self, baseline: float, window_size: int = 5, drop_threshold: flo
        self.baseline = baseline
        self.window_size = window_size
        self.drop_threshold = drop_threshold
        self.scores: list[float] = []
        self.already_rolled_back = False

    def record_score(self, score: float) -> bool:
        self.scores.append(score)
        if self.already_rolled_back:
            return False

        # your turn: same not-enough-data guard and percent-drop check as
        # should_rollback – but on success, also set self.already_rolled_back
        # to True before returning True
        ...
```

Fill in the rest of `record_score` yourself, then feed the same simulated score sequence through a `RollbackMonitor` instance and confirm `rollback_to_previous()` now gets called exactly once.

Compare all 3 of your finished versions against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate are the same core check — a plain function with no memory of its own, which fires every single time the condition is still true. Advanced wraps that same check in a small class that remembers whether it already fired, turning "keeps calling rollback on every bad request forever" into "calls rollback exactly once per decline" — the behavior a real rollback trigger actually needs.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)